GENERALITAT VALENCIANA
Conselleria d'Educació, Cultura i Esport

IES JUAN DE GARAY
C/ Juan de Garay, 25 46017 Valencia
Tel: 961 20 60 45 Fax: 961 20 60 46
46012963@gva.es www.juandegaray.es

GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN Y FORMACIÓN PROFESIONAL

Unión Europea
Fondo Social Europeo
El FSE invierte en tu futuro

UD 06.3PRO. JAVASCRIPT 3B

LM
Ciclos Formativos Grado Superior

Pascual Ligero
pa.ligero2@edu.gva.es
2025js/2026
Versión:260321.1222

Licencia



Reconocimiento – NoComercial – CompartirIgual (by-nc-sa): No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:

⚠ Importante

⚛ Interesante

ÍNDICE DE CONTENIDO

1. Módulos (ES Modules: import js/ export) 5
1.1 Estructura de Archivos Recomendada 5
1.2 Ejemplos de Código 5
1.3 Errores Típicos 6
1.4 Mini-reto (10 min) 6
2. Manejo de Errores (try js/ catch js/ finally) 7
2.1 Ejemplos de Código 7
2.1.1 Lanzar tus propios errores (throw) 7
2.2 Errores Típicos 8
2.3 Mini-reto (5 min) 8
3. Operadores Modernos (Clean Code) 8
3.1 Nullish Coalescing (??) 8
3.2 Optional Chaining (?.) 8
3.3 Asignación Lógica (||=, &&=, ??=) 9
3.3.1 Mini-reto (5 min) 9
4. Persistencia (Client-side) 9
4.1 localStorage vs sessionStorage 9
4.2 El truco del JSON 10
4.3 Errores Típicos 10
4.4 Mini-reto (10 min) 10
5. Web Components (El futuro nativo) 10
5.1 Mini-reto (10 min) 11
6. Cheat Sheet 11
7. Preguntas Rápidas (Vjs/F) 11
8. Checklist de Habilidades 12
9. Ejercicios Finales 12
9.1 Nivel 1: Calentamiento (Sintaxis) 12
9.2 Nivel 2: Intermedio (Lógica + Storage) 13
9.3 Nivel 3: Boss Level (Mini-Apps Modulares) 13
10. 10 prácticas finales 14
10.1 El Dashboard de Cripto "Resiliente" (Arquitectura + Error Handling) 14
10.2 Tienda de "Skins" con Carrito Persistente 14
10.3 Buscador de Películas con Historial (Set + Storage) 15
10.4 Gestor de Tareas "Kanban" (GroupBy + Módulos) 15
10.5 Componente Nativo <github-user> (Web Components) 16
10.6 App del Tiempo con Configuración de Usuario (Optional Chaining) 16
10.7 Validador de Formularios Modular (Throw Error) 17
10.8 "Infinite Scroll" de Fotos (Asyncjs/Await + DOM) 17
10.9 El "Spotify" Local (Audio API + Clases) 17
10.10 Generador de Excusas con "Copy to Clipboard" (Clean Code) 18
11. ¿Cómo entregarlo? 18
12. Bibliografía 19

UD06.3PRO. JAVASCRIPT 3B

1. MÓDULOS (ES MODULES: IMPORT JS/ EXPORT)

Nivel PRO!. Te dejo el siguiente módulo opcional por si quieres aprender mas.

El Problema: Antes tenías un index.html con 20 etiquetas <script> o un solo script.js de 5,000 líneas imposible de leer.

La Solución: Dividir tu código en pequeños archivos especializados (uno para la API, otro para la interfaz, otro para utilidades) y conectarlos entre sí.

1.1 Estructura de Archivos Recomendada

Imagina esta carpeta en tu proyecto:

```
./proyecto
   ├── index.html
   ├── js/main.js  (El cerebro, coordina todo)
   ├── js/api.js   (Se encarga de los fetch)
   └── js/ui.js    (Se encarga de pintar en el HTML)
```

1.2 Ejemplos de Código

Paso 1: El HTML (El punto de entrada)

Necesitas type="module" para activar este superpoder.

HTML

```
<body>
      <h1>App Modular<js/h1>
      <div id="app"><js/div>
      <script type="module" src="js/main.js"><js/script>
<js/body>
```

Paso 2: Exportar funciones (Nombrado vs Default)

JavaScript

```
// api.js
// Exportación nombrada (puedes tener muchas)
export const API_URL = "https://api.ejemplo.com";

export async function traerDatos() {
```

```
    return ["Dato 1", "Dato 2"];
}

// Exportación por defecto (solo una por archivo, ideal para la función principal)
export default function saludar() {
    console.log("¡Hola desde la API!");
}
```

Paso 3: Importar y usar

JavaScript

```
// main.js
// Importamos lo que necesitamos. ¡Fíjate en el .js/ y la extensión .js!
import saludarDefault, { API_URL, traerDatos } from '.js/api.js';

console.log(`Conectando a: ${API_URL}`);
saludarDefault(); // "¡Hola desde la API!"

const datos = await traerDatos();
console.log(datos);
```

1.3 Errores Típicos

- **Error CORS al abrir el HTML directo:** Si abres el archivo con doble clic (file://...), el navegador bloquea los módulos por seguridad.
    - **Corrección:** Usa un servidor local. En VS Code, instala la extensión "Live Server" y dale a "Go Live".
- **Olvidar la extensión:** import ... from '.js/api' fallará.
    - **Corrección:** Siempre pon la extensión: import ... from '.js/api.js'.
- **Rutas relativas:** Olvidar el punto inicial.
    - **Corrección:** Siempre empieza con ./ (carpeta actual) o ../ (carpeta anterior).

1.4 Mini-reto (10 min)

Crea 2 archivos: math.js y app.js.

- En math.js exporta una función sumar(a, b).
- En app.js impórtala, úsala e imprime el resultado.
- Conéctalo a un HTML y mira la consola.

---

2. MANEJO DE ERRORES (TRY JS/ CATCH JS/ FINALLY)

El Problema: Si tu código falla (ej: JSON.parse de algo roto), la web "explota" y se detiene todo el script.

La Solución: "Capturar" el error para que la app siga viva y mostrar un mensaje bonito al usuario.

2.1 Ejemplos de Código

JavaScript

```
const jsonRoto = '{"nombre": "Ana", "edad": }'; // Falta el valor de edad

try {
    // Zona peligrosa: Intentamos hacer algo que puede fallar
    console.log("Intentando leer datos...");
    const usuario = JSON.parse(jsonRoto); // ¡BOOM! Aquí falla
    console.log("Éxito:", usuario.nombre); // Esto NUNCA se ejecuta

} catch (error) {
    // Zona segura: Si falla, caemos aquí
    console.error("¡Ups! Hubo un error:", error.message);
    // Podemos avisar al usuario visualmente
    alert("Los datos vinieron corruptos.");

} finally {
    // Zona neutra: Se ejecuta SIEMPRE (haya error o no)
    console.log("Limpieza terminada.");
}
```

2.1.1 Lanzar tus propios errores (throw)

A veces el código no falla técnicamente, pero lógicamente sí (ej: edad negativa).

JavaScript

```
function registrarUsuario(edad) {
    if (edad < 18) {
        // Nosotros decidimos que esto es un error
        throw new Error("Debes ser mayor de edad");
    }
    return "Registro exitoso";
}

try {
```

```
    registrarUsuario(15);
} catch (e) {
    console.log(e.message); // "Debes ser mayor de edad"
}
```

2.2 Errores Típicos

- **Catch vacío:** catch (e) { }. **Corrección:** ¡Nunca! Al menos haz un console.error o no sabrás qué pasó.
- **Confundir error de red con error de lógica:** fetch no lanza error en 404, pero tú quizás quieras hacer throw si no encuentras el usuario.

2.3 Mini-reto (5 min)

Crea una función dividir(a, b). Si b es 0, lanza un error "No se puede dividir por cero". Envuélvela en un tryjs/catch y prueba con dividir(10, 0).

3. OPERADORES MODERNOS (CLEAN CODE)

Escribir menos código, más legible y más seguro.

3.1 Nullish Coalescing (??)

El "Salvavidas de Nulos". Se usa para valores por defecto.

- **Diferencia con ||**: El || falla con 0 o "" (los considera falsos). El ?? solo salta si es null o undefined.

JavaScript

```
const altura = 0; // 0 es una altura válida

// Problema con ||
const alturaFinalMal = altura || 100; // Devuelve 100 (¡Mal! ignoró el 0)

// Solución con ??
const alturaFinalBien = altura ?? 100; // Devuelve 0 (Correcto)
```

3.2 Optional Chaining (?.)

El "Detector de Minas". Accede a propiedades profundas sin miedo a que explote si algo intermedio no existe.

JavaScript

```
const usuario = { nombre: "Luis" }; // No tiene dirección

// Sin ?. (Explosión)
// console.log(usuario.direccion.calle); // Error: Cannot read properties of undefined

// Con ?. (Seguridad)
console.log(usuario?.direccion?.calle); // undefined (No explota)
console.log(usuario?.saludar?.()); // También sirve para funciones opcionales
```

3.3 Asignación Lógica (||=, &&=, ??=)

Atajos para "Si cumple esto, asígnale aquello".

JavaScript

```
let config = { tema: null };

// ??= "Si es nulojs/undefined, asígnale esto"
config.tema ??= "oscuro";
console.log(config.tema); // "oscuro"

// ||= "Si es falso (0, '', null...), asígnale esto"
let contador = 0;
contador ||= 10; // Cuidado, 0 es falso, así que asigna 10.
```

3.3.1 Mini-reto (5 min)

Tienes const user = { settings: { theme: "" } }.

Usa ?? para guardar en una variable tema el valor de user.settings.theme o por defecto "light".

(Ojo: como es string vacío, || fallaría, pero ?? lo respetará).

---

4. PERSISTENCIA (CLIENT-SIDE)

Guardar datos en el navegador para que no se pierdan al recargar.

4.1 localStorage vs sessionStorage

- **localStorage**: Persistente. Cierras el navegador, apagas la PC, vuelves mañana... y los datos siguen ahí.
- **sessionStorage**: Volátil. Cierras la pestaña y adiós datos.

4.2 El truco del JSON

El storage SOLO guarda **Texto (Strings)**. Si quieres guardar un objeto o array, tienes que convertirlo.

JavaScript

```
const carrito = [{ id: 1, producto: "Pizza" }];

// 1. GUARDAR (Convertir a String)
localStorage.setItem("miCarrito", JSON.stringify(carrito));

// 2. RECUPERAR (Convertir a Objeto)
const datosGuardados = localStorage.getItem("miCarrito");
// Usamos ?? [] por si es la primera vez y no hay nada (null)
const carritoRecuperado = JSON.parse(datosGuardados) ?? [];

console.log(carritoRecuperado[0].producto); // "Pizza"
```

4.3 Errores Típicos

- **Guardar "[object Object]":** Pasa si olvidas JSON.stringify.
- **Leer null:** Si intentas hacer JSON.parse(localStorage.getItem("noExiste")), devuelve null. Si luego haces .push, explota. Siempre usa ?? [] o ?? {}.

4.4 Mini-reto (10 min)

Crea un input y un botón "Guardar". Al escribir y guardar, mételo en localStorage. Recarga la página y haz que el input aparezca relleno con lo que escribiste.

---

5. WEB COMPONENTS (EL FUTURO NATIVO)

Crear tus propias etiquetas HTML (como <mi-boton>) sin React ni Angular.

JavaScript

```
// Definimos la clase de nuestro componente
class TarjetaUsuario extends HTMLElement {
    constructor() {
        super(); // Siempre llamar a super() primero
    }

    // Se ejecuta cuando el elemento entra en el HTML
    connectedCallback() {
        // Leer atributos
```

```
        const nombre = this.getAttribute("nombre") || "Visitante";

        // Renderizar (this.innerHTML es el contenido de la etiqueta)
        this.innerHTML = `
            <div style="border: 1px solid #ccc; padding: 10px; border-radius: 8px;">
                <h3>Hola, ${nombre} 👋<js/h3>
                <p>Bienvenido a mi componente nativo.<js/p>
            <js/div>
        `;
    }
}

// Registramos la etiqueta (debe tener un guion "-")
customElements.define("tarjeta-usuario", TarjetaUsuario);
```

Uso en HTML:

HTML

```
<tarjeta-usuario nombre="Carlos"><js/tarjeta-usuario>
<tarjeta-usuario nombre="Ana"><js/tarjeta-usuario>
```

5.1 Mini-reto (10 min)

Crea un componente <boton-alerta mensaje="Hola"> que sea un botón HTML normal, pero que al hacer click lance un alert con el texto que le pases en el atributo mensaje.

---

6. CHEAT SHEET

- **Módulos:** type="module", import { x } from '.js/file.js', export default.
- **Errores:** try { ... } catch(e) { ... } finally { ... }.
- **Nullish (??):** Solo salta con null o undefined.
- **Optional (?.):** obj?.prop devuelve undefined si obj no existe.
- **Storage:** setItem (guardar), getItem (leer). Siempre JSON.stringify js/ parse.
- **Web Components:** class X extends HTMLElement, connectedCallback, customElements.define.

7. PREGUNTAS RÁPIDAS (VJS/F)

1. Los módulos funcionan abriendo el archivo HTML directamente (file://). (Vjs/F)

2. export default permite exportar varias cosas del mismo archivo. (Vjs/F)
3. JSON.parse lanza un error si el string no es un JSON válido. (Vjs/F)
4. El operador ?? considera el 0 como un valor nulo. (Vjs/F)
5. localStorage borra los datos al cerrar la pestaña. (Vjs/F)
6. sessionStorage comparte datos entre diferentes pestañas. (Vjs/F)
7. obj?.prop evita errores si obj es null. (Vjs/F)
8. Los Web Components necesitan una librería externa para funcionar. (Vjs/F)
9. El método connectedCallback se ejecuta cuando el componente se añade al DOM. (Vjs/F)
10. throw new Error() detiene la ejecución del script si no hay un tryjs/catch. (Vjs/F)

8. CHECKLIST DE HABILIDADES

- [ ] Sé separar mi código en archivos .js e importarlos.
- [ ] Sé capturar errores para que mi web no se rompa.
- [ ] Sé diferenciar cuándo usar ?? y cuándo usar ||.
- [ ] Sé guardar y recuperar objetos complejos en localStorage.
- [ ] He creado mi primera etiqueta HTML personalizada.

9. EJERCICIOS FINALES

Restricciones: Usa módulos (type="module"), constjs/let y nada de frameworks.

9.1 Nivel 1: Calentamiento (Sintaxis)

1. **El Seguro de Vida (?.):** Tienes un array de usuarios, algunos con geo.lat y otros sin geo. Recórrelos e imprime la latitud o "Sin datos" usando optional chaining.
2. **Valores por Defecto (??):** Crea una función que reciba config. Usa ?? para establecer config.velocidad a 10 si no viene. Prueba pasando 0 (debe quedarse en 0).
3. **JSON Seguro:** Intenta hacer JSON.parse de un string mal formado dentro de un tryjs/catch. Si falla, imprime "JSON Inválido" en consola.

4. **Asignación Lógica:** Tienes let usuario = { nombre: "Ana" }. Usa ??= para añadirle rol: "invitado" solo si no tiene rol.

9.2 Nivel 2: Intermedio (Lógica + Storage)

5. **Contador Persistente:** Crea un botón que aumente un contador. Guarda el valor en localStorage. Al recargar la página, el contador debe seguir donde estaba.
6. **Sesión Temporal:** Haz un input "Nombre de usuario". Guárdalo en sessionStorage. Muestra un saludo "Hola, [nombre]". Cierra la pestaña y ábrela: el saludo debe haber desaparecido.
7. **Refactor Clean Code:** Toma un bloque de código antiguo lleno de if (obj && obj.prop) y reescríbelo usando ?. y ??.
8. **Gestor de Errores Fetch:** Haz una función asíncrona que use fetch. Si la URL está mal, captura el error de red. Si la respuesta es 404, lanza tú un error (throw). Muestra ambos casos en consola.
9. **Validador Modular:** Crea validaciones.js con funciones (esEmail, esMayorEdad). Impórtalas en main.js y valida un formulario falso.

9.3 Nivel 3: Boss Level (Mini-Apps Modulares)

10.Lista de Tareas Pro (Persistencia + Módulos):

- storage.js: Funciones guardarTareas(array) y leerTareas().
- ui.js: Función renderTareas(array) que limpie y pinte el DOM.
- main.js: Coordina todo. Al iniciar, lee del storage y pinta. Al añadir, guarda y repinta.

11.**Componente <user-card>:**

- Crea un Web Component que reciba username.
- En su connectedCallback, haz un fetch a https://jsonplaceholder.typicode.comjs/users?username=... (simulado o filtrado).
- Pinta el nombre y el email dentro del componente.
- Usa el componente 3 veces en el HTML con distintos nombres.

12.Theme Switcher Definitivo:

- Botón para alternar Clarojs/Oscuro.
- Guarda la preferencia en localStorage.
- Al cargar la página (antes de pintar nada), lee el storage y aplica la clase dark al body para evitar parpadeos.
- Usa operadores lógicos (tema ??= 'light') para la configuración inicial.

¡Felicidades! Si dominas esto, tu JavaScript ya es de nivel profesional.

Aquí tienes el **"Examen Final Práctico"**.

10. 10 PRÁCTICAS FINALES

están diseñadas para que **dejes de escribir scripts sueltos** y empieces a construir **pequeñas aplicaciones** con arquitectura profesional.

La regla de oro para todas: **Usa Módulos (type="module")**. Nada de un solo archivo index.js de 300 líneas. Divide y vencerás.

10.1 El Dashboard de Cripto "Resiliente" (Arquitectura + Error Handling)

El Reto: Un panel que carga precios de criptomonedas. Si una API falla, intenta con otra.

- **Estructura (M6):** api.js (lógica de fetch), ui.js (pintar), main.js (coordinar).
- **Lógica (M5 + M6):**
  - En api.js, crea una función getBitcoinPrice().
  - Usa **Promise.any()** con dos APIs distintas (ej: CoinGecko y Binance) para obtener el precio de la que responda primero.
  - Usa **tryjs/catch** para manejar el caso de que ambas fallen.
- **DOM (M4):** Pinta el precio y la fuente ("Datos de Binance").
- **Pro Tip (M6):** Usa ?? para mostrar un precio por defecto ("---") si todo falla, y ?. para acceder a los datos anidados del JSON sin miedo.

10.2 Tienda de "Skins" con Carrito Persistente

El Reto: Simular un e-commerce donde el carrito sobrevive si cierras la pestaña.

- **Datos (M5):** Usa https://fakestoreapi.comjs/products (simula que son skins).

- **Persistencia (M6):**
    - Crea storage.js: Funciones saveCart(cart) y getCart().
    - Usa localStorage + JSON.stringifyjs/parse.
- **Lógica (M3):**
    - Al cargar la página, recupera el carrito con getCart() ?? [].
    - Calcula el total con .reduce().
- **DOM (M4):** Renderiza productos y carrito. Al recargar (F5), los productos del carrito deben seguir ahí.

10.3 Buscador de Películas con Historial (Set + Storage)

El Reto: Buscar películas y guardar las últimas 5 búsquedas sin repetir.

- **API:** OMDB API o simulada.
- **Lógica (M3 + M6):**
    - Array historial. Al buscar, haz historial.unshift(termino).
    - Usa new Set() para evitar duplicados ("Batman", "Batman"...).
    - Guarda en localStorage.
- **DOM (M4):**
    - Muestra una lista "Búsquedas recientes". Al clickar una, vuelve a lanzar la búsqueda.
- **Clean Code (M6):** Usa termino ||= "Matrix" (Logical Assignment) para buscar algo por defecto si el input está vacío.

10.4 Gestor de Tareas "Kanban" (GroupBy + Módulos)

El Reto: Un tablero con 3 columnas: "Pendiente", "En curso", "Hecho".

- **Estructura:** tasks.js (estado y funciones puras), drag.js (lógica opcional de arrastre), main.js.
- **Lógica (M3):**
    - Array de objetos: { id, titulo, estado: 'todo' | 'doing' | 'done' }.
    - Usa **Object.groupBy** (ES2024) para separar las tareas por estado antes de pintarlas.

- **Persistencia (M6):** Guarda el estado en localStorage.
- **DOM (M4):**
    - Renderiza 3 columnas.
    - Al clickar una tarea, cambia su estado cíclicamente (todo -> doing -> done) y repinta.

10.5 Componente Nativo <github-user> (Web Components)

El Reto: Crear una etiqueta HTML propia que se pueda reusar 5 veces en la misma página.

- **M6 (Web Components):**
    - class GithubCard extends HTMLElement.
    - En connectedCallback, lee el atributo user="...".
    - Haz el fetch (M5) a la API de GitHub dentro del componente.
- **Shadow DOM (Opcional):** Intenta que los estilos CSS de la tarjeta no afecten al resto de la web.
- **Uso:** En el HTML solo pones:

  HTML

  ```
  <github-user user="midudev"><js/github-user>
  <github-user user="wesbos"><js/github-user>
  ```

10.6 App del Tiempo con Configuración de Usuario (Optional Chaining)

El Reto: Mostrar el clima, permitiendo al usuario elegir °C o °F y guardar esa preferencia.

- **API:** OpenMeteo (gratis y no requiere key).
- **Configuración (M6):**
    - Objeto config = { unit: 'celsius', theme: 'dark' }.
    - Usa ??= para inicializar la config del usuario si no existe en localStorage.
- **Lógica (M3 + M6):**
    - La respuesta de la API es compleja. Usa **Optional Chaining (?.)** masivamente: data?.current_weather?.temperature.

- **DOM (M4):** Un botón de "Settings" que guarde la preferencia en Storage y recargue la vista.

10.7 Validador de Formularios Modular (Throw Error)

El Reto: Un sistema de validación reutilizable.

- **Módulos (M6):**
  - validators.js: Exporta funciones (isEmail, isStrongPass). Si la validación falla, **lanza un error** (throw new Error("Email inválido")).
- **Main:**
  - Importa las funciones.
  - Usa un bloque **tryjs/catch** al enviar el formulario.
  - Si capturas un error, muestra el error.message en un div rojo debajo del input correspondiente.

10.8 "Infinite Scroll" de Fotos (Asyncjs/Await + DOM)

El Reto: Cargar fotos a medida que bajas, gestionando el estado de carga.

- **API:** https://picsum.photosjs/v2js/list?page=X&limit=10
- **Lógica (M5):**
  - Variable pagina = 1. Función asíncrona cargarMas().
  - Usa finally (M6) para quitar el spinner de "Cargando..." tanto si hay éxito como si hay error.
- **DOM (M4):**
  - Detecta cuando el usuario llega al final del scroll (window.innerHeight + window.scrollY >= document.body.offsetHeight).
  - Lanza la petición y añade (append) las nuevas fotos.

10.9 El "Spotify" Local (Audio API + Clases)

El Reto: Un reproductor de música funcional.

- **Estructura (M6):**
  - class Player: Métodos play(), pause(), next().
  - songs.js: Exporta un array de objetos (título, url mp3).

- **Lógica (M2 + M3):**
    - currentSongIndex. Métodos para avanzar usando el módulo (%) para volver al principio si acaba.
- **DOM (M4):**
    - Barra de progreso (input range) que se mueva sola.
    - Lista de reproducción donde la canción actual tenga la clase .active.

10.10 Generador de Excusas con "Copy to Clipboard" (Clean Code)

El Reto: Generar frases aleatorias combinando arrays y copiarlas.

- **Módulos:**
    - data.js: Arrays quien, queHizo, donde.
    - generator.js: Función que combina usando Math.random y **Template Strings**.
- **DOM (M4):**
    - Botón "Generar".
    - Botón "Copiar": Usa la API moderna navigator.clipboard.writeText(texto).
- **M6 (Operadores):**
    - Muestra un mensaje "¡Copiado!" que desaparezca a los 2 segundos.
    - Usa asignación lógica para contar cuántas excusas has generado en esta sesión (contador ||= 0; contador++).

11. ¿CÓMO ENTREGARLO?

Crea una carpeta por proyecto. Oblígate a usar index.html + js/main.js.

¡Si logras hacer el **#1 (Cripto)**, el **#4 (Kanban)** y el **#5 (Web Component)**, ya tienes nivel para optar a ofertas Junior reales!

12. BIBLIOGRAFÍA

Aquí tienes los enlaces originales correspondientes a esos archivos, organizados por autor, para que puedas consultar la fuente directa:

1. Documentación Oficial y Guías (Base Teórica)

Gran parte de la estructura técnica y las definiciones precisas (especialmente del Módulo 1 y 6) provienen de la documentación de Google Developers.

- **Web.dev - Learn JavaScript:** https://web.devjs/learnjs/javascriptjs/

2. Midudev (JavaScript Moderno y ES2023js/24)

La parte de "Modern Syntax", novedades como Object.groupBy, toSorted, asyncjs/await y buenas prácticas actuales se basan en su contenido.

- **Canal de YouTube:** midudev
- **Cursosjs/Web:** midu.dev

3. MoureDev (Fundamentos Sólidos)

Las explicaciones sobre tipos de datos, estructuras y la lógica inicial para principiantes se apoyan en su curso.

- **Curso JavaScript desde Cero:** Ver en YouTube
- **Web:** mouredev.com

4. SoyDalto (Nivel Junior y DOM)

Muchos de los conceptos sobre el DOM (Módulo 4), historia de JS y flujo de ejecución provienen de su curso nivel Junior.

- **Curso de JavaScript (Nivel Junior):** Ver en YouTube

5. Sergie Code (Enfoque Práctico)

Los ejemplos didácticos y el enfoque paso a paso para principiantes absolutos.

- **Curso Gratis JavaScript:** Ver en YouTube